Self-Guided Walkthrough

FinOps Attribution Platform · Sai Sanjay Gogineni

This walkthrough takes about 20 minutes. It shows how the project handles common problems in cloud cost attribution: billing in different formats from different providers, upstream schema changes, restated and late-arriving bills, shared costs with no tag, and the question Finance always asks: *do these numbers add up?*

Each step tells you what to run, what you should see, and where the logic lives.

Just want to see the output? Open `docs/sample_report.html` in a browser. It's the report from a full run, including the restatement in step 3, and needs no setup.

---

0. Setup (5 minutes)

Requirements

- Python 3.11+ (macOS or Linux)
- *Optional:* Java 17+ and Maven, for the Scala Spark job (step 8)
- *Optional:* Terraform 1.6+, to validate the infrastructure code

```
unzip finops-attribution-platform.zip && cd finops-attribution-platform
python3 -m venv .venv && source .venv/bin/activate
make setup          # installs Python dependencies
make data           # downloads the FOCUS sample (~9 MB) from the FinOps Foundation's
GitHub
```

If `make data` fails (for example, a corporate proxy or SSL certificate issue), download `FOCUS-1.0/focus_sample_100000.csv.gz` from https://github.com/FinOps-Open-Cost-and-Usage-Spec/FOCUS-Sample-Data and put it in `data/landing/`.

The data. This is the FinOps Foundation's FOCUS 1.0 sample: 100,000 anonymized, real billing line items from AWS, Microsoft Azure, Oracle Cloud and a small Google Cloud slice. FOCUS is the open billing standard that all major providers now export. The dollar amounts are scaled down by the Foundation, so the ratios and behaviour matter more than the absolute numbers.

---

1. Run the whole pipeline (2 minutes)

```
make pipeline
```

This runs, in order: ingest → telemetry → dbt build → reconcile → benchmark → report.

You should see (abridged):

```
[land] loaded batch 76e9158659d40cde: 100,000 rows, billed=1426.922368,
effective=914.599955, duplicates dropped=0
[telemetry] pages=25 rows=240,750 retries=3 ['HTTP 503', 'HTTP 429', 'HTTP 429']
Done. PASS=26 WARN=1 ERROR=0 SKIP=0 TOTAL=27
[reconcile] run ... -> PASSED
[benchmark] parquet_column_and_partition_pruning   scanned=0.07 MB  reduction=99.3%
```

Open `report/finops_report.html` in a browser and keep it open. The steps below point to sections of it.

The one dbt **WARN** is intentional. It's the data-completeness check (step 5) flagging months that are only partly present in the sample.

---

2. A bad upstream file is blocked, not loaded (2 minutes)

The problem. A provider or API renames a field, or starts sending costs as text (`"$1,234.56"`). A typical loader infers types, turns those values into NULLs, and succeeds. Spend then appears to drop, and nobody notices until month end.

```
make drift-demo
```

You should see:

```
[contract] ERROR PROBABLE_RENAME: required column 'EffectiveCost' missing; found
'Effective_Cost'
[contract] ERROR TYPE_DRIFT: BilledCost expected numeric, 2000 bad values (e.g. '$0.03')
[land] BLOCKED: batch quarantined at data/quarantine/_drift_demo_focus.csv
[drift-demo] loader exited with code 2 -> pipeline halted, lake untouched
```

How it works. Every column is read as text first, then checked against a data contract: required columns, expected types, null limits and known values. A missing column is compared with the new columns to spot a likely rename. Additive columns are allowed and produce a warning.

Look at: `ingestion/contracts/focus_v1_contract.yaml` , `ingestion/validate_contract.py`

---

3. Restated and late-arriving bills (3 minutes)

The problem. Cloud providers re-deliver the current month several times, then revise it after the invoice for credits and refunds. Each delivery is the whole month. If you append it, you double count. Exports can also contain duplicate lines.

```
make restatement-demo
```

This builds a restated AWS September file from the real data: about 1% of lines with corrected costs, 25 late-arriving lines and 10 exact duplicates. It then loads it and re-runs dbt and reconciliation.

You should see:

```
[land] loaded batch d705afbe279354df: 94,376 rows, ... duplicates dropped=10
[land] restatement: replaced partition AWS/2024-09 (was batch 76e9158659d40cde)
[restatement-demo] AWS 2024-09 effective cost: 304.32 -> 346.12 (rows 94,351 -> 94,376)
...
[reconcile] run ... -> PASSED
```

What happened:

- Duplicates were dropped on load; when one line item appears twice, the latest version wins.
- Only the **AWS / 2024-09** partition was replaced. Azure and Oracle were not touched.
- The dbt fact table is **incremental by provider and month** (`delete+insert`), so only that month was rebuilt.
- Reconciliation still ties out, overall and for every provider and month.

In the report, see **Batch history and restatements**.

Look at: `ingestion/load_raw.py` , `transform/models/marts/fct_cost_allocated.sql` (top of file)

---

4. Turning messy tags into owners, and splitting shared cost (4 minutes)

Messy tags. In the real data the owner tag appears as `business_unit` , `org` , `" org"` (with a leading space) and `CostCenter` . A governed alias table maps them to one owner, so the mapping can change without changing code. **Look at:** `transform/seeds/tag_key_aliases.csv`

Shared and untagged cost. Spend without an owner tag is pooled and split by a rule for each category:

Category	Split by
Compute	each team's **CPU core-hours** (telemetry API)
Networking	each team's **request volume** (telemetry API)
Storage, databases, other	each team's share of tagged spend in that category
Credits, tax, adjustments	each team's share of total spend, *not* usage

If a rule's driver is missing for a month, allocation falls back to spend share, and then to an explicit `UNALLOCATED` bucket. Every row records which method produced it, so a disputed number can be traced to its rule. In the report, see **How every dollar was attributed** and **Shared-cost drivers**.

Look at: `transform/seeds/shared_cost_rules.csv`, `transform/models/intermediate/int_allocation_weights.sql`

Telemetry over REST. Service usage comes from an observability API (Datadog-style). The client handles:

- **Pagination** with a guard: if the API repeats a cursor, the client stops instead of looping forever.
- **Retries** on 429s, 5xx errors and network errors, with `Retry-After` support and backoff.
- **Fail-fast** on other 4xx errors.
- **Raw pages saved before parsing**, so a parser fix can be replayed without calling the API again.

You saw `retries=3` in step 1: the mock deliberately returns one 503 and two 429s.

> **Note:** the observability API here is a **local mock with synthetic data**. No public dataset pairs real bills with matching service metrics. The client, the data contract and the allocation logic are real; pointing them at a live API means setting `DD_API_KEY` / `DD_APP_KEY` and adapting the endpoint.

Look at: `ingestion/telemetry/fetch_telemetry.py`, `tests/test_telemetry_client.py`

---

5. Can Finance trust it? Completeness and reconciliation (2 minutes)

```
make reconcile
```

Reconciliation checks that three totals match: the **source files** (control totals recorded when each file was loaded), the **warehouse**, and the **allocated chargeback**. It runs overall and for every provider and month. Every run writes to `audit.reconciliation_runs`, and a dbt test fails the build if allocation creates or loses money.

Completeness. Report section **Data completeness** shows days with usage against days in each month. In this sample, AWS has 30 of 30 days, Azure 19 of 30 and Oracle 15 of 30. Those are flagged as warnings, because a partial month is the most common reason a chargeback looks too low. It also counts late-arriving lines: usage from an earlier month billed in this one.

Look at: `reconciliation/reconcile.py`, `transform/models/marts/mart_data_completeness.sql`

---

6. Measuring storage and query savings (1 minute)

```
make benchmark
```

The same Finance question ("AWS September cost by service") runs against three layouts:

Layout	Bytes scanned	Reduction
gzip CSV, as delivered	9.17 MB	—
Parquet, needed columns only	0.09 MB	99.0%
Parquet, needed columns + one partition	0.07 MB	99.3%

Bytes scanned is what Athena bills. The Parquet figures are read from file metadata, not estimated, and the script first checks that all three layouts return the same answer. Local query latency drops from about 380 ms to about 3 ms.

Look at: `scripts/benchmark_storage.py`

---

7. Commitments (1 minute)

Report section **Commitments**. It shows how much eligible compute is covered by Savings Plans or Reservations, how much commitment went unused (waste), and a recommended commitment sized to the **10th percentile of daily on-demand spend**. Committing to that floor means the commitment is fully used even on quiet days. The discount rate is a placeholder setting (`assumed_commitment_discount` in `transform/dbt_project.yml`), not live pricing.

Look at: `transform/models/marts/mart_commitment_recommendation.sql`

---

8. The Spark / Scala ingestion job (optional, about 3 minutes; needs Java 17+ and Maven)

```
make scala-test     # compiles and runs 7 ScalaTest tests in Spark local mode
make scala-run      # runs the job on the sample, then cross-checks it against the Python
loader
```

`make scala-run` **should end with:**

```
[scala-ingest] AWS/2024-09    94,351         304.32
[scala-ingest] Microsoft/2024-09     4,964     587.279955
...
[scala-ingest] matches Python loader: True
```

The Scala job does the same steps as the local loader:

- **Contract checks** in a single Spark aggregation.
- **Duplicate removal** with a window function.
- **Dynamic partition overwrite**, so a restated month replaces only itself.
- `maxRecordsPerFile` to avoid both swarms of tiny files and single huge files.

It targets Spark 3.5 and Scala 2.12, which is AWS Glue 5.0 / EMR compatible; Spark itself is provided by the cluster.

Look at: `ingestion/spark-scala/src/main/scala/com/finops/ingest/`

---

9. Orchestration: Airflow and Dagster (2 minutes)

The same pipeline is defined twice, to show both styles:

- **Airflow**, `orchestration/airflow/dags/finops_attribution_dag.py` : a daily DAG with retries, an SLA and a Datadog event on failure.
- **Dagster**, `orchestration/dagster/definitions.py` : software-defined assets with a **blocking reconciliation check**, so the report cannot publish if the totals don't tie out.

```
make dagster        # opens the Dagster UI at http://localhost:3000; "Materialize all" runs
the pipeline
```

---

10. Production shape

Concern	Local (this repo runs it)	Production equivalent (included)
Ingestion	DuckDB loader	Scala Spark job / Glue PySpark job (`ingestion/glue/`)
Lake + query	Partitioned Parquet on disk	S3 + Glue Catalog (partition projection) + Athena workgroup with a scan cap
Reference data	dbt seeds	Aurora Serverless v2
Warehouse	DuckDB	Snowflake (`prod` target in `transform/profiles.yml` ; fact clustered by period and owner)
Infrastructure	—	Terraform in `infra/terraform` (validates; not deployed)
CI	`make test`	GitHub Actions: Python + Scala tests, dbt build, restatement and drift regressions, Terraform validate
Consumers	HTML report	Read-only REST API, `make api` → `http://localhost:8000/docs`

Tests at a glance

```
make test           # 26 pytest tests: contract gate, REST client, restatement, pipeline
outputs, API, Dagster wiring
make transform      # 27 dbt nodes: models, seeds and data tests (1 intentional
completeness warning)
make scala-test     # 7 ScalaTest tests
```

Known limitations

- **Telemetry is simulated** (see step 4). **The restatement in step 3 is generated from the real data**; it is not an actual AWS re-delivery.
- The public sample's Google Cloud slice is only 2 rows.
- The sample is row-sampled, so hourly data is sparse; commitment sizing uses daily buckets.
- Savings estimates use a configurable discount rate, not live pricing.
- Terraform is validated but not deployed. The Scala job is tested in Spark local mode, not on a cluster.

Where to start reading the code

1. `transform/models/marts/fct_cost_allocated.sql` : the core allocation logic
2. `ingestion/load_raw.py` : contract, deduplication and restatement handling
3. `reconciliation/reconcile.py` : how the totals are proven
4. `ingestion/telemetry/fetch_telemetry.py` : REST ingestion
5. `ingestion/spark-scala/src/main/scala/com/finops/ingest/FocusIngestJob.scala` : the Spark version

Data source: FinOps Foundation, *FOCUS Sample Data* (FOCUS 1.0), https://github.com/FinOps-Open-Cost-and-Usage-Spec/FOCUS-Sample-Data